Experiment 5 — Client-Server Communication Using TCP (Matrix Classification)

Server program (server.c)

```
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <sys/socket.h>
#include <netinet/in.h>

int main() {
    int server_fd, client_fd, n, a[20][20];
    int upper = 1, lower = 1, diagonal = 1;
    struct sockaddr_in addr;
    char result[100];

    server_fd = socket(AF_INET, SOCK_STREAM, 0);
    addr.sin_family = AF_INET;
    addr.sin_port = htons(9002);
    addr.sin_addr.s_addr = INADDR_ANY;
    bind(server_fd, (struct sockaddr *)&addr, sizeof(addr));
    listen(server_fd, 5);

    printf("Server waiting for client...\n");
    client_fd = accept(server_fd, NULL, NULL);
    recv(client_fd, &n, sizeof(n), 0);
    if (n < 1 || n > 20) {
        close(client_fd);
        close(server_fd);
        return 1;
    }
    recv(client_fd, a, n * n * sizeof(int), 0);

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (i > j && a[i][j] != 0) upper = 0;
            if (i < j && a[i][j] != 0) lower = 0;
            if (i != j && a[i][j] != 0) diagonal = 0;
        }
    }

    if (diagonal)
        strcpy(result, "Diagonal Matrix");
    else if (upper)
        strcpy(result, "Upper Triangular Matrix");
    else if (lower)
        strcpy(result, "Lower Triangular Matrix");
    else
        strcpy(result, "Normal Matrix");

    send(client_fd, result, strlen(result) + 1, 0);
    printf("Matrix type: %s\n", result);
    close(client_fd);
    close(server_fd);
    return 0;
}
```

Client program (client.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

int main() {
    int sockfd, n, a[20][20];
    struct sockaddr_in addr;
    char result[100];

    sockfd = socket(AF_INET, SOCK_STREAM, 0);
    addr.sin_family = AF_INET;
    addr.sin_port = htons(9002);
    inet_pton(AF_INET, "127.0.0.1", &addr.sin_addr);
    connect(sockfd, (struct sockaddr *)&addr, sizeof(addr));

    printf("Enter matrix order: ");
    scanf("%d", &n);
    if (n < 1 || n > 20) {
        printf("Invalid order\n");
        close(sockfd);
        return 1;
    }
```

```
    srand(1);
    printf("Matrix:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            a[i][j] = rand() % 50 + 1;
            printf("%d ", a[i][j]);
        }
        printf("\n");
    }

    send(sockfd, &n, sizeof(n), 0);
    send(sockfd, a, n * n * sizeof(int), 0);
    int r = recv(sockfd, result, sizeof(result) - 1, 0);
    if (r > 0) {
        result[r] = '\0';
        printf("Matrix type: %s\n", result);
    }

    close(sockfd);
    return 0;
}
```

Terminal commands

```
gcc server.c -o server
gcc client.c -o client
```

Run the server in Terminal 1; run the client in Terminal 2.

```
Terminal 1:
./server

Terminal 2:
./client
```

Sample output

```
Server waiting for client...
Matrix type: Normal Matrix

Enter matrix order: 2
Matrix:
34 37
28 16
Matrix type: Normal Matrix
```

Experiment 7 — Multi-user Chat Server Using TCP

Server program (server.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <signal.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <netinet/in.h>

int main() {
    int server_fd, client_fd, n;
    struct sockaddr_in addr;
    char msg[1024];

    signal(SIGCHLD, SIG_IGN);
    server_fd = socket(AF_INET, SOCK_STREAM, 0);
    addr.sin_family = AF_INET;
    addr.sin_port = htons(9002);
    addr.sin_addr.s_addr = INADDR_ANY;
    bind(server_fd, (struct sockaddr *)&addr, sizeof(addr));
    listen(server_fd, 5);

    printf("Chat server waiting for clients...\n");
    while (1) {
        client_fd = accept(server_fd, NULL, NULL);
        if (client_fd < 0) continue;

        if (fork() == 0) {
            close(server_fd);
            while (1) {
                n = recv(client_fd, msg, sizeof(msg) - 1, 0);
                if (n <= 0) break;
                msg[n] = '\0';
                printf("Client message: %s", msg);
                send(client_fd, msg, n, 0);
                if (strncmp(msg, "exit", 4) == 0) break;
            }
            close(client_fd);
            exit(0);
        }
        close(client_fd);
    }

    close(server_fd);
    return 0;
}
```

Client program (client.c)

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

int main() {
    int sockfd, n;
    struct sockaddr_in addr;
    char msg[1024];

    sockfd = socket(AF_INET, SOCK_STREAM, 0);
    addr.sin_family = AF_INET;
    addr.sin_port = htons(9002);
    inet_pton(AF_INET, "127.0.0.1", &addr.sin_addr);

    if (connect(sockfd, (struct sockaddr *)&addr, sizeof(addr)) < 0) {
        perror("Connection failed");
        close(sockfd);
        return 1;
    }
    printf("Connected to server\n");

    while (1) {
        printf("Enter message (exit to quit): ");
        if (fgets(msg, sizeof(msg), stdin) == NULL) break;
        send(sockfd, msg, strlen(msg), 0);

        n = recv(sockfd, msg, sizeof(msg) - 1, 0);
        if (n <= 0) break;
        msg[n] = '\0';
        printf("Server: %s", msg);
```

```
        if (strncmp(msg, "exit", 4) == 0) break;
    }

    close(sockfd);
    return 0;
}
```

Terminal commands

```
gcc server.c -o server
gcc client.c -o client
```

Run the server in Terminal 1; run the client in Terminal 2.

```
Terminal 1:
./server

Terminal 2:
./client
```

Sample output

```
Chat server waiting for clients...
Client message: Hello
Client message: exit

Connected to server
Enter message (exit to quit): Hello
Server: Hello
Enter message (exit to quit): exit
Server: exit
```

Experiment 6 — Client-Server Communication Using UDP (Abbreviation Expansion)

Server program (server.c)

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>

void replace(char *str, const char *old, const char *new) {
    char temp[2048];
    char *pos;
    while ((pos = strstr(str, old)) != NULL) {
        strcpy(temp, pos + strlen(old));
        *pos = '\0';
        strcat(str, new);
        strcat(str, temp);
    }
}

int main() {
    int sockfd, n;
    struct sockaddr_in addr, client;
    socklen_t len;
    char msg[2048], tbuf[2048];

    sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    addr.sin_family = AF_INET;
    addr.sin_port = htons(9002);
    addr.sin_addr.s_addr = INADDR_ANY;
    bind(sockfd, (struct sockaddr *)&addr, sizeof(addr));

    printf("UDP server waiting...\n");
    len = sizeof(client);
    n = recvfrom(sockfd, msg, sizeof(msg) - 1, 0,
                 (struct sockaddr *)&client, &len);
    if (n > 0) {
        msg[n] = '\0';
        replace(msg, "tbh", "to be honest");
        replace(msg, "ig", "I guess");
        replace(msg, "tbf", "to be fair");
        replace(msg, "atm", "at the moment");
        replace(msg, "irl", "in real life");
        replace(msg, "lol", "laughing out loud");
        replace(msg, "asap", "as soon as possible");
        replace(msg, "omg", "oh my God");
        replace(msg, "ttyl", "talk to you later");
        replace(msg, "idk", "I don't know");
        replace(msg, "nvm", "never mind");
        sendto(sockfd, msg, strlen(msg), 0,
               (struct sockaddr *)&client, len);
        printf("Translated sentence: %s\n", msg);
    }

    close(sockfd);
    return 0;
}
```

Client program (client.c)

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

int main() {
    int sockfd, n;
    struct sockaddr_in server;
    socklen_t len;
    char msg[2048];

    sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    server.sin_family = AF_INET;
    server.sin_port = htons(9002);
    inet_pton(AF_INET, "127.0.0.1", &server.sin_addr);

    printf("Enter a sentence: ");
    fgets(msg, sizeof(msg), stdin);
    msg[strcspn(msg, "\n")] = '\0';

    sendto(sockfd, msg, strlen(msg), 0,
```

```
            (struct sockaddr *)&server, sizeof(server));

    len = sizeof(server);
    n = recvfrom(sockfd, msg, sizeof(msg) - 1, 0,
                 (struct sockaddr *)&server, &len);
    if (n > 0) {
        msg[n] = '\0';
        printf("Formal English: %s\n", msg);
    }

    close(sockfd);
    return 0;
}
```

Terminal commands

```
gcc server.c -o server
gcc client.c -o client
```

Run the server in Terminal 1; run the client in Terminal 2.

```
Terminal 1:
./server

Terminal 2:
./client
```

Sample output

```
UDP server waiting...
Translated sentence: to be honest I don't know what to do laughing out loud

Enter a sentence: tbh idk what to do lol
Formal English: to be honest I don't know what to do laughing out loud
```

Experiment 8 — Time Server Application Using UDP

Server program (server.c)

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <time.h>
#include <sys/socket.h>
#include <netinet/in.h>

int main() {
    int sockfd, n;
    struct sockaddr_in server, client;
    socklen_t len;
    char msg[100], tbuf[100];
    time_t now;
    struct tm *t;

    sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    server.sin_family = AF_INET;
    server.sin_port = htons(1234);
    server.sin_addr.s_addr = INADDR_ANY;
    bind(sockfd, (struct sockaddr *)&server, sizeof(server));

    printf("Time server started...\n");
    while (1) {
        len = sizeof(client);
        n = recvfrom(sockfd, msg, sizeof(msg) - 1, 0,
                     (struct sockaddr *)&client, &len);
        if (n < 0) continue;

        msg[n] = '\0';
        printf("Request: %s\n", msg);
        if (strcmp(msg, "stop") == 0) break;

        now = time(NULL);
        t = localtime(&now);
        strftime(tbuf, sizeof(tbuf), "%c", t);
        sendto(sockfd, tbuf, strlen(tbuf) + 1, 0,
               (struct sockaddr *)&client, len);
    }

    close(sockfd);
    return 0;
}
```

Client program (client.c)

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

int main() {
    int sockfd, n;
    struct sockaddr_in server;
    socklen_t len;
    char msg[100], tbuf[100];

    sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    server.sin_family = AF_INET;
    server.sin_port = htons(1234);
    inet_pton(AF_INET, "127.0.0.1", &server.sin_addr);

    while (1) {
        printf("Enter request (stop to quit): ");
        if (fgets(msg, sizeof(msg), stdin) == NULL) break;
        msg[strcspn(msg, "\n")] = '\0';

        sendto(sockfd, msg, strlen(msg) + 1, 0,
               (struct sockaddr *)&server, sizeof(server));
        if (strcmp(msg, "stop") == 0) break;

        len = sizeof(server);
        n = recvfrom(sockfd, tbuf, sizeof(tbuf) - 1, 0,
                     (struct sockaddr *)&server, &len);
        if (n <= 0) break;

        tbuf[n] = '\0';
        printf("Server time: %s\n", tbuf);
    }

    close(sockfd);
```

```
    return 0;
}
```

Terminal commands

```
gcc server.c -o server
gcc client.c -o client
```

Run the server in Terminal 1; run the client in Terminal 2.

```
Terminal 1:
./server

Terminal 2:
./client
```

Sample output

```
Time server started...
Request: time
Request: stop

Enter request (stop to quit): time
Server time: Sat Oct 24 10:15:30 2026
Enter request (stop to quit): stop
```